

第三节 JavaScript

一、JavaScript简介

HTML给网络信息传递带来了极大的便利，但是，单纯地使用HTML代码进行编辑具有一定的缺陷，它只能静态地显示一些已经提供的固有资源，缺乏必要的用户与客户端、用户和服务器之间的交互，这使网页的使用友好度受到一定限制。

JavaScript的出现，满足了用户与客户端互动的需求，使得页面和用户之间不仅仅只是显示和浏览的关系，而是实现了动态的可交互式的表达能力。

参考书：

1. **JavaScript入门经典（第7版）**，**Phil Ballard**著，人民邮电出版社，2019
2. **《JavaScript从入门到精通（第3版）》**（*Steve Suehring* 著），清华大学出版社，2014

SAMS
Teach
Yourself

- 全球销量逾百万册的系列图书
- 连续十余年打造的经典品牌
- 直观、循序渐进的学习教程
- 掌握关键知识的最佳起点
- “Read Less, Do More” (精读多练) 的教学理念
- 以示例引导读者完成最常见的任务

Pearson

异步图书
www.it-ebooks.com

每章内容针对初学者精心设计, **1** 小时轻松阅读学习,
24 小时彻底掌握关键知识

每章 **案例与练习题** 助你轻松完成常见任务,
通过 **实践** 提高应用技能, 巩固所学知识

JavaScript

入门经典 (第7版)

[美] 菲尔·巴拉德 (Phil Ballard) 著
李强 译

人民邮电出版社
POSTS & TELECOM PRESS

人民邮电出版社
POSTS & TELECOM PRESS

软件开发精英大讲堂
JAVASCRIPT/HTML5/JS

十年锤炼
打造经典

JavaScript

从入门到精通

24 小时微课视频讲解

明日科技 编著

- ✓ 技术资源库
- ✓ 实例资源库
- ✓ 测试题库系统
- ✓ 面试资源库
- ✓ PPT 电子课件

(第3版)

循序渐进, 实战讲述

基础知识 ◀ 核心技术 ▶ 高级应用 ▶ 项目实战
214 个应用实例, 46 个实践练习, 1 个项目案例

海量资源, 可查可练

除本书配套的 24 小时视频讲解外, 根据学习顺序, 资源包还额外配备如下海量开发资源库:

技术资源库 (800 页技术参考文档) ▶ 实例资源库 (400 个实例) ▶ 测试题库系统 (138 道测试题) ▶ 面试资源库 (369 个面试真题)

在线解答, 高效学习

清华大学出版社

1. 什么是 JavaScript?

- JavaScript 可被直接嵌入HTML页面;
- JavaScript 是一种脚本语言;
- JavaScript 是一种解释性语言;
- JavaScript 是一种基于对象和事件驱动的解释性语言;
- JavaScript 是一种动态类型语言;

JavaScript能够做什么？

- (1) 写入 **HTML** 输出;
- (2) 对事件作出反应;
- (3) 改变 **HTML** 内容;
- (4) 改变 **HTML** 样式;
- (5) 验证输入;

.....

说明：JavaScript与Java是两种不同的语言。

Java是SUN公司推出的面向对象的程序设计语言，特别适合于Internet应用程序开发；

而JavaScript是Netscape公司的产品，其目的是为了扩展Netscape浏览器功能，而开发的一种可以嵌入Web页面中的解释性语言。

2.在HTML中包含JavaScript

与在HTML中插入CSS样式表不同，网页设计者可以选择在HTML页面的任意位置添加JavaScript代码。添加方法有两种：

- 在HTML文件当中使用<script>标记直接编写JavaScript代码；

```
< script >  
    JavaScript代码;  
    .....  
< /script>
```

- 从外部的JavaScript文件导入到HTML文件中

```
<script src="JavaScript文件地址"></script>
```

<i> <body> 中的 JavaScript

在本例中，JavaScript 会在页面加载时向 HTML 的 **<body>** 写文本：

```
<!DOCTYPE html>
<html>
<body>
  <script>
    document.write("<h1>This is a heading</h1>");
    document.write("<p>This is a paragraph</p>");
  </script>
</body>
</html>
```

<ii> <head> 中的 JavaScript 函数

可以在HTML文档中放入不限数量的脚本。脚本可位于HTML的<body> 或 <head> 部分中，或者同时存在于两个部分中。通常的做法是把函数放入 <head> 部分中，从而让文档的其他部分能够调用其中的内容。

```
<!DOCTYPE html>
<html>
<head>
<script>
    function myFunction()
    { document.getElementById("demo").innerHTML="My First JavaScript Function"; }
</script>
</head>
<body>
<h1>My Web Page</h1>
<p id="demo">A Paragraph</p>
<button type="button" onclick="myFunction()">Try it</button>
</body>
</html>
```

<iii>外部的 JavaScript

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
  <script src="myScript.js">
```

```
  </script>
```

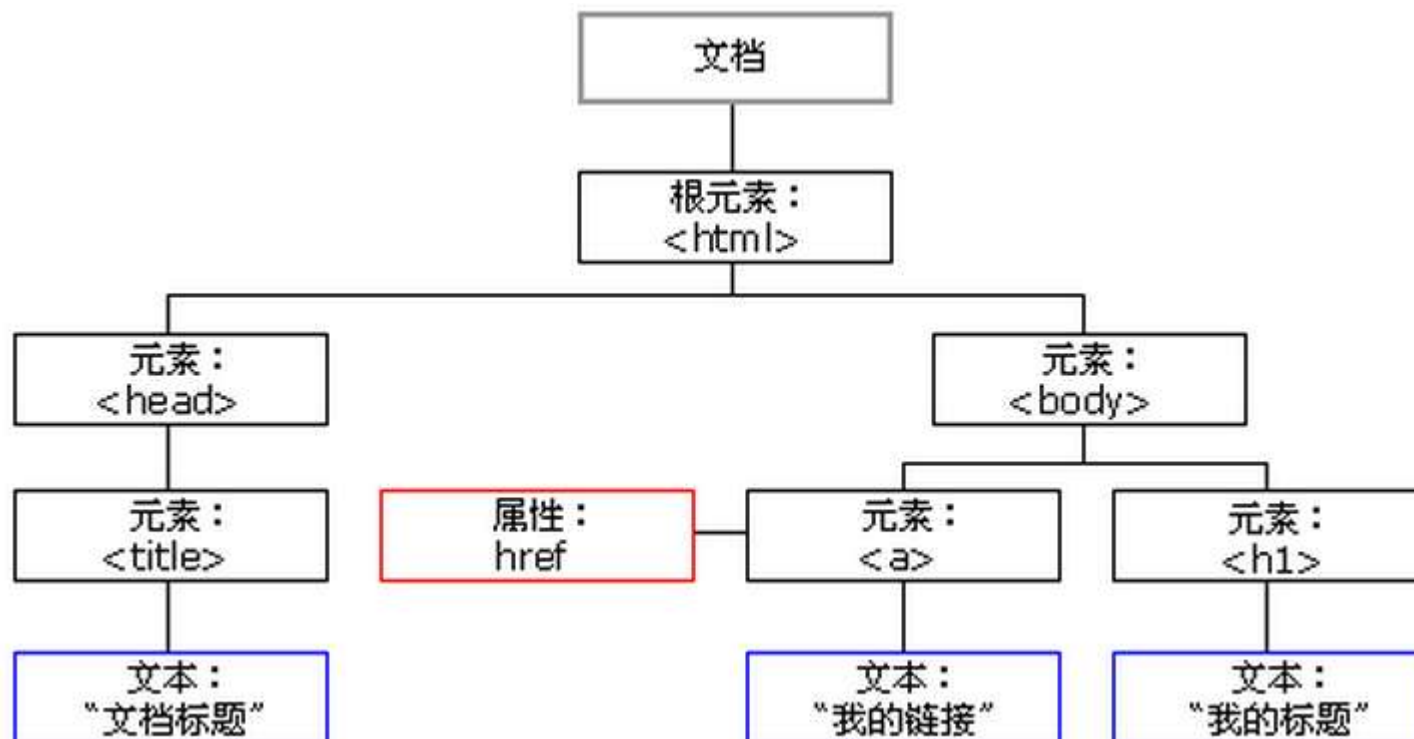
```
</body>
```

```
</html>
```


二、HTML DOM（文档对象模型） （Document Object Model）

- 当网页被加载时，浏览器会创建页面的文档对象模型；
- 浏览器每次加载和显示页面时，都在内存里创建页面及其全部元素的一个内部表示体系，也就是DOM。在DOM里，页面的元素具有一个逻辑化、层级化的结构，就像相互关联的父对象和子对象组成了一个树形的结构。
- 通过DOM，可访问HTML文档的所有元素；

1.HTML DOM 树



通过可编程的文档对象模型，**JavaScript** 获得了足够的能力来创建动态的 **HTML**。

- **JavaScript** 能够改变页面中的所有 **HTML** 元素；
- **JavaScript** 能够改变页面中的所有 **HTML** 属性；
- **JavaScript** 能够改变页面中的所有 **CSS** 样式；
- **JavaScript** 能够对页面中的所有事件做出反应；

(1)通过 id 查找 HTML 元素

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<p id="intro">Hello World!</p>
```

```
<p>本例演示 <b>getElementById</b> 方法! </p>
```

```
<script>
```

```
  x=document.getElementById("intro");
```

```
  document.write('<p>id="intro" 的段落中的文本是: ' + x.innerHTML + '</p>');
```

```
</script>
```

```
</body>
```

```
</html>
```

说明：JavaScript 由 web 浏览器来执行。在这种情况下，浏览器将访问 id="intro" 的 HTML 元素，并把它的内容（innerHTML）输出。

通过 id 查找 HTML 元素。

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<p id="intro">Hello World!</p>
```

```
<p>本例演示 <b>getElementById</b> 方法! </p>
```

```
<script>
```

```
  x=document.getElementById("intro");
```

```
  document.write('<p>id="intro" 的段落中的文本是: ' +  
                                     x.innerHTML + '</p>');
```

```
</script>
```

```
</body>
```

```
</html>
```

三、JavaScript语言概览

1. ECMAScript 10

ECMAScript是一种由Ecma国际(前身为欧洲计算机制造商协会, European Computer Manufacturers Association)通过ECMA-262标准化的脚本程序设计语言。这种语言在万维网上应用广泛, 它往往被称为JavaScript或JScript, 所以它可以理解是JavaScript的一个标准, 但实际上后两者是ECMA-262标准的实现和扩展。

2015 年 6 月 17 日，ECMA 国际组织发布了 **ECMAScript 的第六版**，该版本正式名称为 **ECMAScript 2015**，但通常被称为 **ECMAScript 6** 或者 **ES6**。自 ES6 以后 ECMAScript 每年发布一次新标准。

说明：

ES6是JavaScript语言的一个重要版本。它引入了许多新特性，如类、模块、箭头函数、let和const声明、迭代器和生成器等。这些新特性使得JavaScript编程更加现代和强大。

2. JavaScript 语句

- JavaScript 是脚本语言，浏览器会在读取代码时，逐行地执行脚本代码；
- JavaScript 对大小写是敏感的；
- 通常我们在每条可执行的语句结尾添加分号（在 JavaScript 中，用分号来结束语句是可选的）；

语句1

语句2

语句1;语句2;

语句1;

语句2;

- 代码注释:

//注释内容

//注释内容

/* 跨越多行
的注释 */

3.JavaScript 变量

- 变量可以用任何方式命名，但我们一般只使用字母、数字、美元符号（\$）和下划线（_）符号开头；
- 变量名称对大小写敏感（y 和 Y 是不同的变量）；
- JavaScript程序员和其他很多程序员一样，习惯使用一种名为“驼峰式命名法”；

如：myName;

小驼峰法：

小驼峰法的意思是：除第一个单词之外，其他单词首字母大写

。如： **studentCount=0;**

这种方式常用于变量命名；

大驼峰法：

相比小驼峰法，大驼峰法把第一个单词的首字母也大写了。

如： **StudentInfomation;**

种命名方式通常用于类名、类型名、方法名等。

- 变量赋值:

```
var netPrice
```

```
netPrice = 8.99;
```

```
var netPrice = 8.99;
```

```
var productName="Leather wallet";
```

```
netPrice = 8.99;
```

- 变量的生命期:

变量的生存周期是指变量在计算机中存在的有效时间。

JavaScript中变量的生存周期有两种:

全局变量和局部变量;

全局变量在主程序中定义，其有效范围从其定义开始，一直到本程序结束为止。局部变量在程序的函数中定义，其有效范围只有在函数之中，当函数结束后，局部变量的生存周期也就结束了。

全局变量和局部变量可以使用相同的名称，但仍然是不同的变量;

4.JavaScript数据类型

- 数据类型：字符串、**数字**、布尔、数组、对象、**null**、**undefined**;
- JavaScript 拥有**动态类型**。这意味着相同的变量可用作不同的类型;

var x; **// x 为 undefined**

var x = 6; **// x 为数字**

var x = "Bill"; **// x 为字符串**

- 字符串可以是引号中的任意文本。您可以使用单引号或双引号:

var carname="Bill Gates";

var carname='Bill Gates';

- **JavaScript** 只有一种数字类型。数字可以带小数点，也可以不带；
- **undefined** 未定义值：表示变量还未赋值；
- **null**：空值

如果试图引用一个没有定义的变量，返回一个空值；

说明：**null**与**undefined**的区别：**null**表示一个变量被赋予了一个空值；**undefined**表示该变量尚未被赋值；

(1) JavaScript 数组:

创建新数组:

```
var cars=new Array();
```

```
var cars=[]; //创建数组的另外一种方便形式
```

初始化数组:

```
cars[0]="Audi";
```

```
cars[1]="BMW";
```

```
cars[2]="Volvo";
```

```
var cars=new Array("Audi","BMW","Volvo");
```

说明: 数组拥有内建的属性 **length**。对于上面的字符串来说, **length** 的值是 3。

(2) JavaScript 对象

- **对象由花括号分隔。**在括号内部，对象的属性以名称和值对的形式 (**name : value**) 来定义。属性由逗号分隔：

```
var person={firstname:"Bill", lastname:"Gates", id:5566};
```

```
var person={  
    firstname : "Bill",  
    lastname : "Gates",  
    id : 5566 };
```

- 对象属性有两种寻址方式：

```
name=person.lastname;
```

```
name=person["lastname"];
```

5.JavaScript 运算符

JavaScript 算术运算符

运算符	描述	例子	结果
+	加	$x=y+2$	$x=7$
-	减	$x=y-2$	$x=3$
*	乘	$x=y*2$	$x=10$
/	除	$x=y/2$	$x=2.5$
%	求余数 (保留整数)	$x=y\%2$	$x=1$
++	累加	$x=++y$	$x=6$
--	递减	$x=--y$	$x=4$

JavaScript 赋值运算符

运算符	例子	等价于	结果
=	x=y		x=5
+=	x+=y	x=x+y	x=15
-=	x-=y	x=x-y	x=5
=	x=y	x=x*y	x=50
/=	x/=y	x=x/y	x=2
%=	x%=y	x=x%y	x=0

用于字符串的 + 运算符：

+ 运算符用于把文本值或字符串变量加起来（连接起来）。

Eg: txt1="What a very";

txt2="nice day"; txt3=txt1+txt2;

对字符串和数字进行加法运算：

- **x=5+5; document.write(x);**
- **x="5"+"5"; document.write(x);**
- **x=5+"5"; document.write(x);**
- **x="5"+5; document.write(x);**

注意：如果把运算符“+”用于一个字符串变量和一个数值变量，JavaScript 会把数值转换为字符串，再把两个字符串连接起来。

■比较运算符

== (值)、!=、>、>=、<、<=

=== 全等 (值和类型)

■逻辑运算符

! && ||

■条件运算符

操作数? 结果1:结果2

■其他运算符

typeof运算符用于返回它的操作数当前容纳的数据类型。

这对于判断一个变量是否已被定义特别有用。

例：应用**typeof**运算符返回当前所容纳的数据类型。

```
<html>
<body>
<script>
  var a=3;
  var b="name";
  var c=null;
  alert("a的类型为"+typeof(a)+ "\nb的类型为" +typeof(b)+ "\nc的类型为" +typeof(c));
</script>
</body>
</html>
```

6. 控制语句

(1) 分支语句

- **if 语句**:只有当指定条件为 **true** 时, 使用该语句来执行代码;
- **if...else 语句**:当条件为 **true** 时执行代码, 当条件为 **false** 时执行其他代码;
- **if...else if....else 语句**:使用该语句来选择多个代码块之一来执行;
- **switch 语句**:使用该语句来选择多个代码块之一来执行;

例子:

```
if (temperature<35) {  
    message="Heating";  
    heatingStatus="on";  
    ...;  
} else {  
    message="Stop heating";  
    heatingStatus="off";  
    ....;  
}
```


(2) 循环语句

JavaScript 支持不同类型的循环：

- **while** - 当指定的条件为 **true** 时循环指定的代码块；
- **do/while** - 同样当指定的条件为 **true** 时循环指定的代码块；
- **for** - 循环代码块一定的次数；
- **for/in** - 循环遍历对象的属性；

while语句:

```
while (expression) {  
    statement  
}
```

do...while语句:

```
do {  
    statement  
} while (expression);
```

For循环:

```
for (initialize; test; increment) {  
    statement  
}
```

例:计算100以内所有奇数的和。

<script>

var sum=0;

for (i=1; i<100; i+=2) {

sum=sum+i;

}

alert("100以内所有奇数的和为:"+sum);

</script>

(3) For/In 循环

JavaScript **for/in** 语句循环遍历对象的属性；利用 **for...in** 循环可以操作任何对象的属性，可以是DOM对象、JavaScript内置对象、或我们创建的对象。

for/in语句的语法：

**for (variable in object)
statement;**

提供了一种遍历对象属性的方法。

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<p>点击下面的按钮，循环遍历对象 "person" 的属性。 </p>
```

```
<button onclick="myFunction()">点击这里</button>
```

```
<p id="demo"></p>
```

```
<script>
```

```
function myFunction()
```

```
{  var x, txt="", person={fname:"Roger",lname:"Tomlinson",age:92};
```

```
    for (x in person)
```

```
    {  txt=txt + person[x];  }
```

```
    document.getElementById("demo").innerHTML=txt;
```

```
}
```

```
</script>
```

```
</body>
```

```
</html>
```

1.请写出以下代码在按钮点击之后的浏览器输出。

```
<!DOCTYPE html>
<html>
<body>
<p>点击下面的按钮，循环遍历对象 "person" 的属性。</p>
<button onclick="myFunction()">点击这里</button>
<p id="demo"></p>
<script>
function myFunction()
{   var x, txt="";

    var people =
    { "programmers": [ { "firstName": "Brett", "lastName": "McLaughlin", "email": "aaaa" }, { "firstName": "Jason",
        "lastName": "Hunter", "email": "bbbb" }, { "firstName": "Elliotte", "lastName": "Harold", "email": "cccc" } ],
      "authors": [ { "firstName": "Isaac", "lastName": "Asimov", "genre": "science fiction" }, { "firstName": "Tad",
        "lastName": "Williams", "genre": "fantasy" }, { "firstName": "Frank", "lastName": "Peretti", "genre":
        "christian fiction" } ],
      "musicians": [ { "firstName": "Eric", "lastName": "Clapton", "instrument": "guitar" }, { "firstName": "Sergei",
        "lastName": "Rachmaninoff", "instrument": "piano" } ]
    }

    for (x in people)
    {   txt=txt+x+" : "+ people[x][1].firstName+'\n';   }

    alert(txt);
}

</script>
</body>
</html>
```

作答

四、JavaScript内部对象

- String对象；
- Array对象；
- Date对象；
- Math对象；
- Boolean对象；
- Number对象；
- RegExp对象；

说明：此外，JavaScript 允许自定义对象；

1. String对象

这样声明一个 JavaScript 变量时：

```
var txt = "Hello World!";
```

txt是String类型

实际上已经创建了一个 JavaScript 字符串对象。字符串对象拥有内建的属性 `length`。字符串对象同时拥有若干个内建的方法。

属性： `var x = txt.length;`

方法： `var x = txt.toUpperCase();`

```
var txt = new string("Hello World!");
```

txt是Object类型

2. Math 对象

- Math对象提供了大量的数学常量和数学函数;
- 在使用Math 对象时, 不能使用new关键字创建对象实例, 直接使用“对象名.成员”的格式来访问其属性或方法;

例: `var nVal=-3.45;`
`var pVal=Math.abs(nVal);`

- Math对象的属性:
- Math对象的方法:

例：随机产生指定位数的验证码。

（1）编写随机产生指定位数的验证码函数`checkCode()`，该函数只有一个参数`digit`，用于指定产生的验证码的位数，返回值为指定位数的验证码。

（2）编写JavaScript自定义函数`deal()`，调用`checkCode()`函数将转换后的字符串显示在指定位置；

（3）在页面添加一个`<div>`标记，将其命名为`result`，用于显示产生的验证码；

```
<!DOCTYPE html>
<html>
<head>
  <title>随机产生指定位数的验证码</title>
</head>
<body>
  <div id="result">产生的验证码: </div>
  <div>=====</div>
  <form>
    <div>请输入要产生的验证码的位数: </div>
    <input type="text" id="digit">&nbsp;&nbsp; (1~15)
    <br/>
    <br/>
    <input type="button" value="生成" onclick="deal()">
    <button onclick="clear()">刷新</button>
  </form>
  <script>
    function checkCode(digit) {                                     //编写随机产生指定位数的验证码函数
      var result = "";
      for (i = 0; i < parseInt(digit); i++) {
        result = result + (Number.parseInt(Math.random() * 10));    }
      return result;          }
    function deal() {
      result.innerHTML = "产生的验证码: " + checkCode(digit.value);  }
    function clear() {
      digit.value = "";
      result.innerHTML = "产生的验证码: ";    }
  </script>
</body>
</html>
```

- **Math.random()**: 返回0..1之间的随机数;
- **Number.parseInt()**函数: 可解析一个字符串, 并返回一个整数;

3. RegExp对象

(1) 正则表达式

- 有的时候，需要一种方法在一段文本中匹配一个较为严格或具体的模式。这就是正则表达式的用武之地；
- 正则表达式是一个特殊的字符串，用来描述搜索模式。在程序中，这样的一种模式可以用于搜索、替换或者管理字符串在一段文本中的出现情况。

□ 正则表达式一些常见的例子：

- 验证用户输入数据的格式，例如E-mail 地址、电话号码或者邮政编码；
- 找到在一篇文章中多次出现的URL 并进行更新；
- 操作HTML 源代码中特殊的标签和属性；
-

(2) 语法

var patt=new RegExp(pattern, modifiers);

或者更简单的方式:

var patt=/pattern/modifiers;

修饰符:

修饰符用于执行区分大小写和全局匹配:

修饰符	描述
i	执行对大小写不敏感的匹配。
g	执行全局匹配（查找所有匹配而非在找到第一个匹配后停止）。
m	执行多行匹配。

(3) RegExp对象方法

方法	描述
<u>exec</u>	检索字符串中指定的值。返回找到的值。
<u>test</u>	检索字符串中指定的值。返回 true 或 false。
<u>toString</u>	返回正则表达式的字符串。

□ **test方法**：将判断以参数传入的字符串是否与正则表达式相匹配，返回值是 **true** 或 **false**;

例1： `var re=new RegExp("JavaScript rules");`
`var str="JavaScript rules";`
`if(re.test(str)) document.writeln("I guess it does rule");`

例2： `var patt1=new RegExp("e");`
`document.write(patt1.test("The best things in life are free"));`

■ **exec()** 方法检索字符串中的指定值。返回值是被找到的值，如果没有发现匹配，则返回 **null**;

例1: `var patt1=new RegExp("e");`
`document.write(patt1.exec("The best things in life are free"));`

可以向 **RegExp** 对象添加第二个参数，以设定检索。例如，如果需要找到所有某个字符的所有存在，则可以使用 **"g"** 参数;

例2: `var patt1=new RegExp("e","g");`
`do { result=patt1.exec("The best things in life are free");`
`document.write(result); }`
`while (result!=null)`

五、JSON格式（JavaScript Object Notation）

JavaScript对象表示法

1.JSON的提出

21世纪初，Douglas Crockford寻找一种简便的数据交换格式，能够在服务器之间交换数据。当时通用的数据交换语言是XML（eXtensible Markup Language），但是Douglas Crockford觉得XML的生成和解析都太麻烦，所以他提出了一种简化格式，也就是JSON。

JSON的规格非常简单，只用一个页面几百个字就能说清楚，而且Douglas Crockford声称这个规格永远不必升级，因为该规定的都规定了。

<?xml version="1.0" encoding="GB2312"?>

XML声明

<订单>

根元素

<顾客>张三</顾客>

子元素

<日期>2009-03-08</日期>

<送货地址>广东汕头

元素内容文本

<明细>

<条目 序号="1">

属性

<商品>康乃馨</商品>

<数量>1</数量>

<单价>5</单价>

</条目>

<条目 序号="2">

<商品>玫瑰</商品>

<数量>1</数量>

<单价>5</单价>

</条目>

</明细>

<备注>送母亲和老婆</备注>

</订单>

树型结构
严谨/易读

2.什么是 JSON?

- **JSON 是轻量级的数据交换格式**
- **JSON 独立于语言***
- **JSON 是“自描述的”且易于理解**

说明*： JSON 的语法是来自 JavaScript 对象符号的语法，但 JSON 格式是纯文本。读取和生成 JSON 数据的代码可以在任何编程语言编写的。

3. JSON语法

JSON 数据的表示方式是一系列成对的参数与值，参数与值由冒号分隔，每对之间以逗号分隔：

"param1":"value1", "param2":"value2", "param3":"value3"

最终这些序列用花括号包围起来，构成表示数据的JSON对象：

```
var jsonObject = {  
    "param1":"value1",  
    "param2":"value2",  
    "param3":"value3"  
}
```

4.JSON实例

(1) JSON 数据：名称和值

JSON 数据的书写方式是名称/值对，类似 JavaScript 对象属性。

(2) 名称/值对

由（双引号中的）字段名+冒号+值构成；

"firstName":"Bill"

JSON 名称需要双引号。JavaScript 名称不需要。

JSON格式示例1:

```
{  
  "name": "中国",  
  "province": [  
    {  
      "name": "黑龙江",  
      "cities": {  
        "city": ["哈尔滨", "大庆"]  
      }  
    }, {  
      "name": "广东",  
      "cities": {  
        "city": ["广州", "深圳", "珠海"]  
      }  
    }, {  
      "name": "台湾",  
      "cities": {  
        "city": ["台北", "高雄"]  
      }  
    }, {  
      "name": "新疆",  
      "cities": {  
        "city": ["乌鲁木齐"]  
      }  
    }  
  ]  
}
```


JSON格式示例2:

```
[  
  {  
    "title":"Java实战经典开发",  
    "edition":3,  
    "author":["smith","尼古拉斯","斯巴达"]  
  },  
  {  
    "title":"Oracle实战经典开发",  
    "edition":3,  
    "author":["smith","尼古拉斯","斯巴达"]  
  },  
  {  
    "title":"Vue实战经典开发",  
    "edition":5,  
    "author":["smith","尼古拉斯","斯巴达"]  
  }  
]
```

JSON 和 JS 对象互转:

- 要实现从JSON字符串转换为JS对象, 使用 `JSON.parse()` 方法:

```
var obj = JSON.parse('{"a": "Hello", "b": "World"}');
```

结果: `{a: 'Hello', b: 'World'}`

- 要实现从JS对象转换为JSON字符串, 使用

`JSON.stringify()` 方法:

```
var JSON = JSON.stringify({a: 'Hello', b: 'World'});
```

结果: `'{"a": "Hello", "b": "World"}'`

六、函数

(1) 基本语法

在JavaScript中，函数是由关键字function、函数名加一组参数以及置于大括号中需要执行的一段代码定义的。定义函数的基本语法如下：

```
function functionName([parameter1,parameter2,...])  
{  
    statements;  
    [return expression;]  
}
```

参数说明：

functionName：必选项。用于指定函数名。在同一个页面中，函数名必须是唯一的，并且区分大小写。

parameter：可选项。用于指定参数列表。

statements：必选项。是函数体，用于实现函数功能的语句。

return expression：可选项。用于返回函数值。

(2) 函数的调用

调用函数只需要使用函数名称（以及一对括号），就可以在需要的地方执行函数的代码。

例：

```
<!DOCTYPE html>
<head>
<script>
    function jisuan()
    {
        var price = document.getElementById("text1").value;
        var number = document.getElementById("text2").value;
        var x = account(price, number);
        document.write("总价是" + x);
    }
    function account(price, number)
    {
        var sum = price * number;
        return sum;
    }
</script>
</head>
<body>
    价格: <input type="text" id="text1" />
    数量: <input type="text" id="text2" />
    <input type="button" id="button1" onclick="jisuan()" value="计算" />
</body>
</html>
```

■ 通过链接调用函数

函数除了可以在响应事件中被调用外，还可以在链接中被调用。在<a>标签中的href属性使用“**javascript: 函数名**”**链接来调用函数**，当用户单击该链接时，相关函数将被执行。下面的代码实现了通过链接调用函数。

```
<!DOCTYPE html>
<head>
  <title>示例</title>
</head>
<script>
  function test()
  {
    alert("我喜欢javascript");
  }
</script>
<body>
  <a href="javascript:test();">test</a>
</body>
</html>
```

(3) 嵌套函数

所谓嵌套函数即在函数内部再定义一个函数，这样定义的优点在于可以使内部函数轻松获得外部函数的参数以及函数的变量等。

语法：

```
<script>
    var outer=10;
    function functionName(parameter1,parameter2) //定义外部函数
    {
        function InnerFunction() //定义内部函数
        {
            somestatements;
        }
    }
</script>
```

参数说明：

(1) **functionName**: 外部函数名称。

(2) **InnerFunction**: 嵌套函数名称。

函数嵌套应用

```
<!DOCTYPE html>
<head>
  <title>嵌套函数的应用</title>
</head>
<H1>这是嵌套函数的应用示例</H1>
<script>
var outter=10;
function add(number1,number2)
{
  function innerAdd()
  {
    alert("参数和为: "+(number1+number2+outter));
  }
  return innerAdd();
}
</script>
<body>
<script type="text/javascript">
add(10,10);
</script>
</body>
</html>
```

(4) 变量作用域

- 函数内部声明的变量只存在于函数的内部;
- 函数内部声明的变量称为局部变量; 函数之外声明的变量称为全局变量;
- 局部变量和全局变量可以使用相同的名称, 但仍为不同的变量;

```
<!DOCTYPE html>
<head>
  <title>变量作用域</title>
</head>
<body>
  <script>
    var a=10;
    var b=10;
    function showVars()
    {
      var a=20;
      b=20;
      return "Local variable 'a' = " + a + " \n Global variable 'b' = " + b;
    }
    var message = showVars();
    alert ( message+ " \n Global variable 'a' = " + a);
  </script>
</body>
</html>
```


！ 使用**this**关键字：

在函数中使用**this** 时，关键字**this** 引用的“拥有”该函数的任何对象，即该函数的父作用域的内容。

```
<!DOCTYPE html>
<head>
  <title>变量作用域</title>
</head>
<body>
<script>
  var a=10;
  var b=10;
  function showVars()
  {
    var a=20;
    b=20;
    return " this.a = " + this.a + "\n Local variable 'a' = " + a + " \n Global variable 'b' = " + b;
  }
  var message = showVars();
  alert ( message+ " \n Global variable 'a' = " + a);
</script>
</body>
</html>
```

```

<!DOCTYPE html>
<head>
  <title>变量作用域</title>
</head>
<body>
<script>
  var a=10;
  var b=10;
  function showVars()
  {
    var a=20;
    b=20;
    return " this.a = " + this.a + "\n Local variable 'a' = " + a + " \n Global variable 'b' = " + b;
  }
  var message = showVars();
  alert ( message+ " \n Global variable 'a' = " + a);
</script>
</body>
</html>

```

➤ 进一步讲解: var 变量定义一用与不用

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<p>++++++++++++++++++++++++++++++++++++</p>
```

```
<script>
```

```
function test2() {  
    document.write(name);  
    name = "test2"  
    document.write(name);  
}
```

```
function test1() {  
    name = "test1";  
    document.write(name);  
}
```

```
function test3() {  
    document.write(this.name);  
    document.write(name);  
    var name = "test3";
```



```
}  
test1();  
test2();  
test3();
```

```
</script>
```

```
</body>
```

```
</html>
```

```
function test3() {  
    var name;           //js声明变量,会在  
                        //用域的最顶部声  
                        //但并没有赋值。  
                        //为undefined  
    document.write(this.name)  
    document.write(name)  
    name = "test";      //此时进行  
}
```

说明：ECMAScript一个有趣的方面，在使用变量之前可不声明，ECMAScript的解释程序遇到未声明过的变量时，会创建一个全局变量，并将其初始化为指定的值。故test1()方法中的无var声明的变量name为全局变量，调用结果为test1。不过如果不能紧密跟踪变量，这样做很危险。**最好的习惯是像使用其他程序设计语言一样，总是声明所有变量。**

(5) let - 块级作用域

函数内部声明的变量只存在于函数的内部；

- 在现代JavaScript (ES6及之后)中，变量是通过let关键字声明的：

```
let i;
```

```
let sum;
```

也可以使用一条let语句声明多个变量：

```
let i, sum;
```

- 声明变量的同时(如果可能)也为其赋予一个初始值是个好的编程习惯：

```
let message = "hello";
```

```
let i=0, j=0, k=0;
```

```
let x=2, y=x*x;
```

- 如果在let语句中不为变量指定初始值，变量也会被声明，但被赋为undefined；
- 在同一个作用域中使用多个let或const声明同一个名字是语法错误；

- 通过`let`和`const`声明的变量和常量具有**块作用域**。这意味着它们只在`let`和`const`语句所在的代码块中有定义。

`JavaScript`函数的函数体是代码块，`if/else`语句的语句体
`while`和`for`循环的循环体都是代码块。

- 粗略地讲，如果变量或常量声明在一对花括号中，那这对花括号就限定了该变量或常量有定义的代码区域。作为`for`、`for/in`或`for/of`循环的一部分声明的变量和常量，以循环体作为它们的作用域，即使它们实际上位于花括号外部。

七、JavaScript 对象

1.创建JavaScript对象

通过 JavaScript，你能够定义并创建自己的对象，创建新对象有两种不同的方法：

- (1) 定义并创建对象的实例（创建直接实例）；
- (2) 使用构造函数来定义对象，然后创建新的对象实例；

(1) 创建直接实例（定义并创建对象的实例）

JavaScript有一个内置对象Object，利用它可以创建一个空白的对象；

例： **var person=new Object();**
person.firstname="John";
person.lastname="Doe";
person.age=50;
person.eyecolor="blue";

a.添加方法:

首先定义一个函数，然后把它附加到person作为方法。

```
function myFunc () {  
    alert(this.firstname);  
}  
person.showInfo = myFunc;
```

注意：在把函数关联到对象时，只使用了函数名称，并不包含括号。这是因为我们是要把函数myFunc()的定义赋予person.showInfo方法。如果使用像下面这样的代码：

```
person.showInfo = myFunc();
```

其作用是让JS执行函数myFunc()，然后把它的返回值赋予person.showInfo。

示例：

```
<!DOCTYPE>
<html>
<head>
<title>Object Oriented Programming</title>
<script>
    var person=new Object();
    person.firstname="John";
    person.lastname="Doe";
    person.age=50;
    person.eyecolor="blue";
    function myFunc () {
        alert(this.firstname);
    }
    person.showInfo = myFunc;
</script>
</head>
<body>
<input type="button" value="Good showInfo Call" onclick="person.showInfo()" />
<input type="button" value="myFunc Call" onclick="myFunc()" />
<input type="button" value="Bad showInfo Call" onclick="showInfo()" />
</body>
</html>
```

b. 匿名函数

前面介绍了一种设置对象方法的方式，就是创建一个单独的函数，然后把它的名称赋予某个方法。现在介绍一种更简单方便的方式。如下：

```
person.showInfo = function() {  
    alert(this.firstname);  
}
```

这种方式不需要给函数命名，所以称为“匿名函数”。

对象初始化:

```
var person = {  
    firstname:"John",  
    lastname:"Doe",  
    age:50,  
    eyecolor:"blue",  
    showInfo:function() {  
        alert(this.firstname);  
    }  
}
```

(2)使用构造函数定义对象

```
function student(name,age,score){  
    this.name=name;  
    this.age=age;  
    this.score=score;  
}
```

创建实例:

```
var stuA=new student(“John”,21,98);  
var stuB=new student(“Sally”,20,100);
```

<!DOCTYPE html>

<html>

<body>

<script>

function student(name,age,score){

this.name=name;

this.age=age;

this.score=score;

}

aStu=new student("John",21,98);

document.write(aStu.name + " is " + aStu.score);

</script>

</body>

</html>

2.使用prototype扩展和继承对象

使用对象的主要优点之一是能够在崭新的环境中重复使用已经编写好的代码。JavaScript提供的机制能够基于已有的对象来修改对象，使其拥有新的方法或属性，甚至可以创建完全崭新的对象。这些技术分别叫作“扩展”和“继承”。

(1) 扩展对象

当一个对象已经实例化之后，如果想使其具有新的方法和属性，可以使用关键字prototype。prototype对象允许迅速地添加属性和方法，然后就可以用于对象的全部实例。使用prototype扩展对象。

我们来扩展前面范例里的对象person，给它添加一个新方法sayHello()。

```
person.prototype.sayHello=function() {  
    alert(this.name + " says hello");  
}
```


示例:

```
<!DOCTYPE>
<html>
<head>
<title>面向对象编程</title>
<script>
    function Person(personName){
        this.name = personName;
        this.info = "Call"+this.name;
        this.showInfo = function() {
            alert(this.info);
        }
    }
    var p1=new Person('Mike');
    var p2=new Person('Li');
    Person.prototype.sayHello = function() {
        alert(this.name + "say hello");
    }
</script>
</head>
<body>
<input type="button" value="Mike" onclick="p1.showInfo()" />
<input type="button" value="Li" onclick="p2.showInfo()" />
<input type="button" value="Hello Mike" onclick="p1.sayHello()" />
<input type="button" value="Hello Li" onclick="p2.sayHello()" />
</body>
</html>
```

注意：JavaScript 中 prototype 与 类

JS中的类与C++中的类非常不一样，虽然语法上有些类似。JS也模拟一些经典的类特性，但是：JS的类和基于原型的继承机制与C++等语言中类和基于类的继承机制有着本质区别。

ES6引入class关键字，但是新增后并未改变JS类基于原型（**prototype**）的本质。

(2) 继承

继承是指从一种对象类型创建另一种对象类型，新对象类型继承老对象类型的属性和方法，还可以可选地添加自己的属性和方法。通过该方式，可以先设计出“通用”的对象类型，然后继承来不断细化他们来得到更特定的类型，这样可以节省很多工作。

JavaScript模拟实现继承的方式也是使用关键字——**prototype**。

```

<!DOCTYPE html>
<html>
<head>
<script>
    function PET() {
        this.animal="";
        this.name="";
        this.setAnimal=function(newAnimal){
            this.animal=newAnimal; }
        this.setName=function(newName){
            this.name=newName; }
    }
    function fish() {
        this.breed=""
        this.setBreed=function(newBreed){
            this.breed=newBreed; }
    }
    fish.prototype=new PET();
</script>
</head>
<body>
<script>
    var myfish=new fish();
    myfish.setName("jack");
    myfish.setBreed("anchovy");
    alert(myfish.name +" is " + myfish.breed);
</script>
</body>
</html>

```

作答

八、JavaScript事件

1.什么是事件(event)？

一些JavaScript程序是从第一行代码开始执行，然后一行一行地执行，在此过程中并不会留意可能发生的任何事情，直到程序完成。然而，我们往往想要**让程序对于在环境中发生的事情做出响应**。例如，这些事情可能是用户点击页面元素，也可能是一幅图像完成了载入。我们将这些情况称为事件。JavaScript能够检测众多的事件并对它们做出响应。常规事件有鼠标事件、键盘事件、DOM对象事件和表单事件。

2.事件处理器

事件处理器是当JavaScript检测到一个特定的事件时所执行的一段代码。如：当用户的鼠标悬停在输入按钮上时，该按钮的颜色改变了；当用户按下P键时，程序的执行暂停了；当页面完成加载时，菜单元素变得可见了。有几种不同的方式添加事件处理器。

3. JavaScript常用的绑定事件方法

- 在DOM元素中直接绑定
- 在JavaScript代码中绑定（动态绑定）
- 绑定事件监听函数

(1) 在DOM元素中直接绑定

JavaScript支持在元素中直接绑定事件，语法为：

<标签 属性列表 onXXX=“事件处理程序”/>

eg:

<input type=“button” onclick=“display()” />

其中：

onXXX 为事件名称。例如，鼠标单击事件 **onclick**，鼠标双击事件 **ondouble**，鼠标移入事件 **onmouseover**，鼠标移出事件 **onmouseout** 等。

结构+样式+行为都绑定在同一个标签中，不利于后期维护。

（2）在JavaScript代码中绑定（动态绑定）

在JavaScript代码中（即<script>标签内）绑定事件可以使JavaScript代码与HTML标签分离，文档结构清晰，便于管理和开发。

在JavaScript代码中绑定事件的语法为：

```
elementObject.onXXX=function() {  
    // 事件处理代码  
}
```

其中：

- elementObject 为DOM对象，即DOM元素。
- onXXX 为事件名称。

例如，为 id="demo" 的按钮绑定一个事件，显示它的 type 属性：

```
<!DOCTYPE html>
```

```
<html>
```

```
<body>
```

```
<input id="demo" type="button" value="点击我，显示 type 属性"/>
```

```
<script>
```

```
    document.getElementById("demo").onclick=function(){  
        alert(this.getAttribute("type"));  
    }
```

```
</script>
```

```
</body>
```

```
</html>
```

(3) 绑定事件监听函数

■ 用 `addEventListener()` 绑定事件监听函数；

■ `addEventListener()`函数语法：

`elementObject.addEventListener(eventName,handle,useCapture)`

参数	说明
<code>elementObject</code>	DOM对象（即DOM元素）。
<code>eventName</code>	事件名称。注意，这里的事件名称没有“ on ”，如鼠标单击事件 click ，鼠标双击事件 doubleclick 等。
<code>handle</code>	事件句柄函数，即用来处理事件的函数。
<code>useCapture</code>	Boolean类型，是否使用捕获，一般用false。

示例演示与讲解：

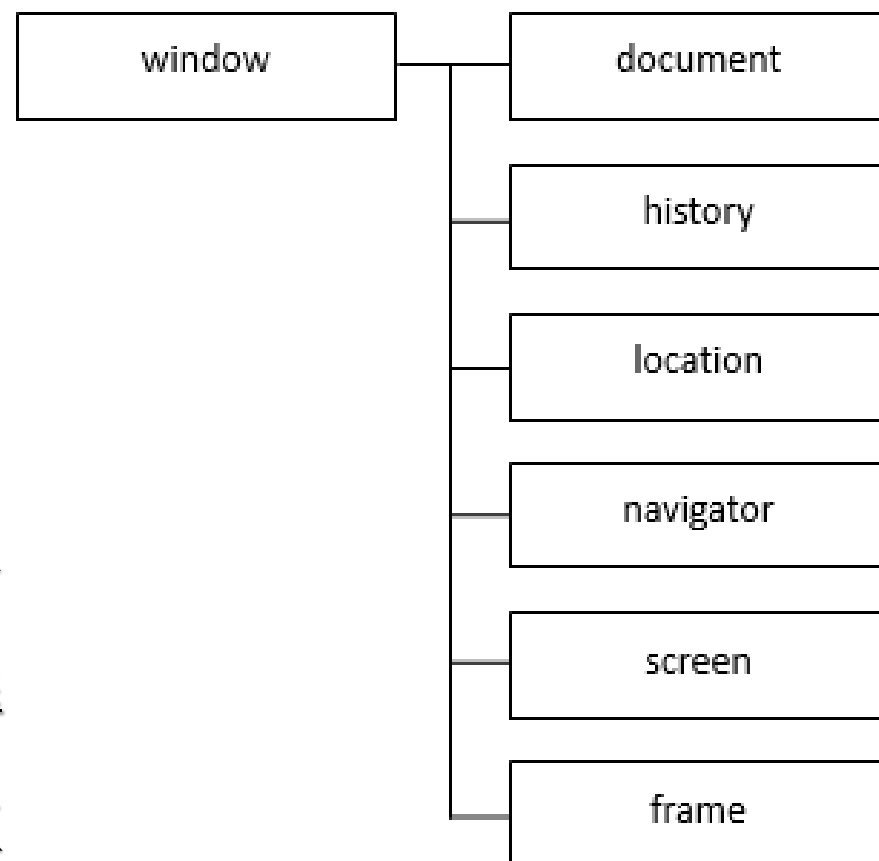
bindevent.html

<u>target</u>	返回触发此事件的元素（事件的目标节点）；
----------------------	----------------------

九、浏览器对象模型（Browser Object Model, BOM）

在实际应用中，常常使用JavaScript操作浏览器窗口以及窗口上的控件，从而实现用户和页面的动态交互功能。浏览器预定义了很多内置对象，这些对象都含有相应的属性和方法，通过这些属性和方法可以控制浏览器窗口及其控件。**客户端浏览器这些预定义的对象统称为浏览器对象，它们按照某种层次组织起来的模型统称为浏览器对象模型。**浏览器对象模型定义了浏览器对象的组成和相互关系，描述了浏览器对象的层次结构。

BOM由多个对象组成，其中代表浏览器窗口的**Window**对象是**BOM**的顶层对象，其他对象都是该对象的子对象——**document、history、location、navigator、screen 及frame**对象。这些对象都含有若干属性和方法，使用这些属性和方法可以操作Web浏览器窗口中的不同对象，控制和访问HTML页面中的不同内容。



浏览器对象模型

window 对象:

在实际应用中，只要打开浏览器，无论是否存在页面，**window** 对象都将被创建。由于**window**对象是所有对象的顶层对象，所以按照对象层次时不必显式地注明**window**对象。

window对象内置了许多方法供用户操作，下面列出最常用的**window**对象的方法，如表所示。

方法名	说明
alert(message)	显示带有一段消息和一个确认按钮的告警框
confirm(question)	显示带有一段消息以及确认按钮和取消按钮的对话框
open(url,name,features,replace)	打开一个新的浏览器窗口或查找一个已命名的窗口
prompt(“提示信息”，默认值)	显示可提示用户输入的对话框
blur()	把键盘焦点从顶层窗口移开
close()	关闭浏览器窗口
focus()	把键盘焦点给予一个窗口
setInterval(code,interval)	按照指定的周期(以毫秒计)来调用函数或计算表达式
setTimeout(code,delay)	在指定的毫秒数后调用函数或计算表达式
clearInterval(intervalID)	取消由setInterval()设置的timeout
clearTimeout(timeoutID)	取消由setTimeout()方法设置的timeout

十、JavaScript库

JavaScript库就是可重用JavaScript代码的集合，让我们的程序只需添加几行代码就能完成复杂的操作。

1.为什么使用库

- 比我们自己编写的代码会更完善。常用的JavaScript库包含了程序员经常会用到的函数。这些库得到了数以千计的下载和评论，证明它们包含的代码经过了完整的测试和调试；
- 吸取其他程序员的思路；
- 利用细致编写的库可以避免跨浏览器时JavaScript可能产生的问题；
-

2. Javascript库能做什么

- 封装DOM方法。 Javascript库可以提供方便的方式来选择和管理页面元素或元素组；
- 动画。利用库只需要很少的代码就能够方便地实现滑动、淡出、晃动、变形、跳动等效果，而且是在多浏览器都能正常运行；
- 拖放。真正跨浏览器实现拖放操作时相当复杂的，使用库可以大大简化这个工作；
- Ajax方面，不必考虑XMLHttpRequest实例化的问题，不必关心回调函数和状态码，就能动态更新页面内容。

3.常见的库

- **Prototype**框架：优势在于**DOM**扩展和**Ajax**处理，在**JSON**支持与创建和继承类方面也做的不错；
- **Dojo**：是个开源工具集，能够简化创建程序 and 用户界面的工作；最新版本不仅支持全部主流浏览器，还支持手机环境；
- **jQuery**：是个小型高效的**Javascript**库，简化了多种开发工作，比如**HTML**文档转换、事件处理、动画和**Ajax**调用；

- **Vue.js:** Vue.js是一套构建用户界面的渐进式框架，采用自底向上增量开发的设计，核心库只关注视图层；通过尽可能简单的 API 实现响应的数据绑定和组合的视图组件；
- **React.js:** 用于构建用户界面的JS库，由Facebook开发和维护，专注于高效、组件化的Web应用开发。其核心特点包括组件化开发、虚拟DOM、声明式编程和跨平台能力；
- **Angular :**

十一、AJAX

Asynchronous JavaScript and XML

(异步的 JavaScript 和 XML)

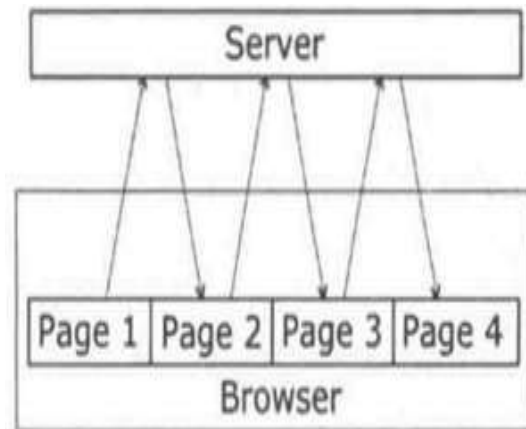
1. AJAX解析

AJAX可以与后台服务器通信，并且**不需要执行页面刷新就能够将结果显示到页面上**。这就使得你能够创建与用户平滑交互的页面。

问题描述:

举例：在填写表单的字段时，我们会根据需要进行编辑，因为知道在最终提交（或保存）之前，数据不会被发送到服务器。

！ 在提交表单或进行保存等操作之后，浏览器要进行屏幕刷新，才能显示由服务器发送的新页面或修改后的页面。



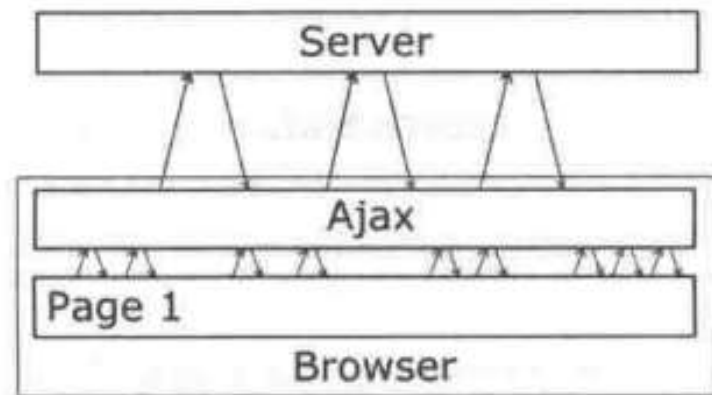
存在的问题:

- 首先，每个新页面或修改页面的加载都会有明显延时，这会严重影响用户对于应用程序“流畅”运行的感觉；
- 而且，即使新页面与前个页面的内容几乎是相同的，每次也都需要加载“整个”页面。站点里很多页面的共同元素，比如标题、导航栏等，可能会在页面数据里占据很大的比例；这种不必要的数据下载会浪费带宽；

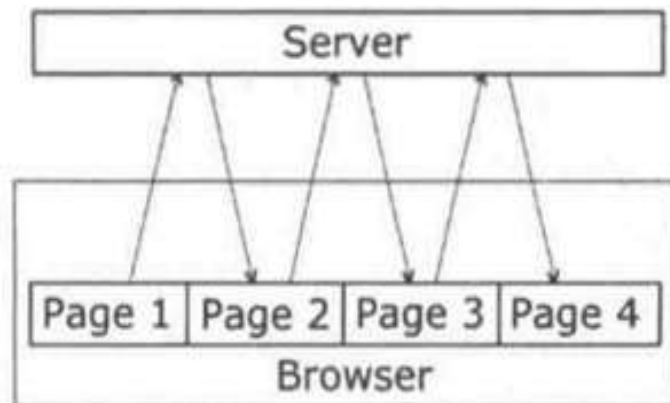
2. AJAX入门

Ajax在Web页面与服务器之间建立了一个额外的“处理层”，这个“处理层”通常称为Ajax引擎或Ajax框架。

Ajax解释来自用户的请求，在后台以异步方式“安静”地处理服务器通信。这意味着对于用户操作，客户端请求与服务器响应不再需要同步一致了；浏览器不会停止响应来等服务器完成对最后一个请求的处理，而是会允许用户在当前页面浏览、单击和输入数据。



Time →
Ajax 客户端/服务器交互



3. XMLHttpRequest对象

■ 为了让Web程序实现异步工作，给服务器发送HTTP请求而不必显示新页面，利用XMLHttpRequest对象就可以实现这种方式。它能够建立与服务器的连接，发送HTTP请求而不需要加载相应的页面。

说明：出于安全的考虑，XMLHttpRequest对象一般只能调用与当前页面同一个域里的URL，而不能直接调用远程服务器。

XMLHttpRequest对象的属性和方法

属性	描述
onreadystatechange	当对象的 readyState 属性改变时，调用哪个事件处理器
readyState	以整数形式反映请求的状态 0=未初始化 1=正在加载 2=加载完成 3=交互 4=完成
responseText	以字符串形式从服务器返回的数据
responseXML	以文档对象形式从服务器返回的数据
status	服务器返回的 HTTP 状态代码
statusText	服务器返回的解释短语
方法	描述
abort()	停止当前请求
getAllResponseHeaders()	以字符串形式返回全部标题
getResponseHeader(x)	以字符串形式返回标题 x 的值
open('method', 'URL', 'a')	指定 HTTP 方法（GET 或 POST）、目标 URL 和处理请求的方式（a=true，默认，表示异步；a=false，表示同步）
send(content)	发送请求。对于 POST 数据是可选的
setRequestHeader('x', 'y')	设置“参数=值”对（x=y），把它赋予与请求一起发送的标题

工作原理：

- 浏览器触发事件（点击、输入、定时）
- JS 创建 XMLHttpRequest 对象
- 向服务器发送异步请求
- 服务器处理并返回数据（通常是 JSON）
- JS 接收数据，局部更新页面